GRUPO 2 · REQUERIMIENTOS FUNCIONALES

Tarjetas de Crédito y Débito

Módulo: Tarjetas, límites, disponible, estado y vencimiento

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 2
Versión	1.0
Fecha	10 de octubre de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST, `/api/v2`) y aiquaa-sandbox-web (front, `/v2`)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training_v2
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el ciclo de vida de una tarjeta: emisión (crédito o débito), consulta, renovación de sus datos (marca y vencimiento), bloqueo y reactivación, cambio del límite de crédito y baja. Incluye el cálculo del disponible de una tarjeta de crédito como límite menos saldo utilizado.

Fuera de alcance: Consumos, pagos del resumen y cualquier operación que modifique el saldo utilizado (ningún endpoint lo cambia); intereses, cargos y cierre de resumen; el vencimiento automático por fecha; y los datos sensibles de la tarjeta (número completo, código de seguridad, PIN), que no existen en el sandbox.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con la API key de su grupo (o con un token JWT de grupo) y automatiza escenarios BDD sobre el módulo de su grupo.
Cliente del banco (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es titular de cuentas, tarjetas, préstamos, beneficiarios, planes de ahorro y depósitos.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo sobre el schema `qa_training_v2`, bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz tipo home banking que consume la API a través del proxy `/api/proxy/v2/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v2/tarjetas	RF-G2-01	Lista tarjetas vigentes, filtradas por titular y/o estado.
POST /api/v2/tarjetas	RF-G2-02	Emite una tarjeta de crédito o de débito.
GET /api/v2/tarjetas/{id}	RF-G2-03	Devuelve el detalle de una tarjeta con su disponible.
PUT /api/v2/tarjetas/{id}	RF-G2-04	Reemplaza marca y fecha de vencimiento.
DELETE /api/v2/tarjetas/{id}	RF-G2-05	Da de baja una tarjeta sin saldo utilizado.
POST /api/v2/tarjetas/{id}/bloquear	RF-G2-06	Bloquea una tarjeta activa.
POST /api/v2/tarjetas/{id}/activar	RF-G2-07	Reactiva una tarjeta bloqueada y no vencida.
PATCH /api/v2/tarjetas/{id}/limite	RF-G2-08	Cambia el límite de una tarjeta de crédito.
POST /api/v2/g2/auth/token	RF-G2-TK	Emite el token JWT del grupo.

2. Precondiciones y acceso

- **Base URL de la API:** `https://aiquaa-sandbox-api.vercel.app`. La documentación interactiva del curso está en `/docs/v2` y la especificación OpenAPI en `GET /api/v2/docs`. La aplicación web está en `https://aiquaa-sandbox-web.vercel.app`.

- Toda request debe autenticarse por una de dos vías: el header `x-api-key` con una clave activa de `public.api_keys`, o el header `Authorization: Bearer <token>` con un JWT de grupo vigente. Si se envían ambos, manda `x-api-key`. Sin credenciales, o con credenciales inválidas, inactivas o vencidas, la respuesta es `401 UNAUTHORIZED` con un mensaje genérico que no indica cuál fue el motivo.
- Las rutas bajo `/api/v2` solo aceptan credenciales del **curso 2**: una API key válida pero del curso 1 recibe `403 FORBIDDEN` con el mensaje "Esta API key pertenece al curso 1 y esta ruta es del curso 2.". El gate es simétrico: una key del curso 2 tampoco sirve en `/api/v1`.
- Cada credencial admite un máximo de **30 requests por minuto** (ventana deslizante, contada por API key o por usuario del token, no por IP). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training_v2` (10 tablas) y se cargan con `scripts/seed-data-v2.sql`: 8 clientes (más un cliente por grupo, usado por el login de token), 12 cuentas, 20 movimientos, 10 tarjetas, 6 préstamos con sus cuotas, 10 beneficiarios, 12 transferencias, 6 planes de ahorro y 6 depósitos. La data inicial es determinística, pero **el sandbox es compartido por los cinco grupos**: los registros sembrados pueden haber sido modificados por otra suite. Un escenario robusto crea sus propios datos en lugar de depender del estado de una fila sembrada.
- **Clientes de soporte:** `GET|POST /api/v2/usuarios` y `GET|PUT|DELETE /api/v2/usuarios/{id}` no pertenecen a ningún grupo y están disponibles para todos; sirven para crear el titular de los productos de cada escenario. `GET /usuarios` acepta el filtro `email`; `POST` exige `nombre`, `email` y `documentoNumero` (email y documento son únicos: repetirlos da `409 CONFLICT`).
- **Sandbox SQL:** `POST /api/v2/sql/select` y `POST /api/v2/sql/update` reciben `{ "sql", "params" }` y permiten consultar o preparar datos directamente sobre las 10 tablas del curso. Admiten un único statement por request, con placeholders `$1..$n`; el `UPDATE` exige cláusula `WHERE`. Son la vía para verificar en base lo que hizo un endpoint, o para forzar un estado que la API no produce.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` hace baja lógica (`activo = false`, o `activa = false` en `cuentas`) y devuelve `204 No Content` sin cuerpo. Una fila dada de baja deja de verse en `GET`, `PUT` y `DELETE`, pero sigue existiendo en la base.
- En la aplicación web el acceso tiene tres capas, en orden: elegir el curso (`/curso`, opción 2), cargar la API key del curso (`/login`, que valida que la key sea del curso elegido) y elegir el cliente con el que se opera (`/auth/login`, por email de un cliente activo). Las pantallas `/v2/usuarios/*` son la excepción a la tercera capa: se pueden usar solo con la API key, porque son el punto de partida para crear un cliente.
- Para emitir una tarjeta se necesita un cliente existente (`usuarioId`); la cuenta asociada (`cuentaId`) es opcional.
- Datos sembrados útiles: tarjetas 1, 3, 4, 7 y 9 son de crédito y nacen `activa`; la 3 y la 9 están casi al tope de su límite; la 4 no tiene consumo; las tarjetas 5 y 8 son de débito y nacen `bloqueada`; la tarjeta 10 nace `vencida` con saldo utilizado. La tarjeta 6 (Amex) vence el 31 de octubre de 2026: su comportamiento cambia según la fecha en que se ejecute la prueba.
- Como ningún endpoint modifica el saldo utilizado, un escenario que necesite una tarjeta con consumo debe usar una sembrada o prepararla con `POST /api/v2/sql/update` sobre la columna `saldo_utilizado` (respetando que no supere el límite).

3. Modelo de datos del módulo

tarjetas

Tarjetas emitidas. `estado` es el estado de negocio y `activo` la marca de baja lógica. El disponible no es una columna: se calcula en cada consulta.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`cuenta_id`	`bigint`, opcional, referencia a `cuentas(id)`
`tipo`	`text`, `credito` o `debito` (por defecto `debito`)
`marca`	`text`, `visa`, `mastercard` o `amex` (por defecto `visa`)
`numero_enmascarado`	`text`, obligatorio; con el formato `** ** NNNN`, generado al azar al emitir. No es único.
`limite_credito`	`numeric(12,2)`, obligatorio, por defecto 0; mayor o igual a cero
`saldo_utilizado`	`numeric(12,2)`, obligatorio, por defecto 0; mayor o igual a cero y nunca mayor al límite
`estado`	`text`, uno de `activa` / `bloqueada` / `vencida`; por defecto `activa`
`fecha_vencimiento`	`date`, obligatoria
`activo`	`boolean`, obligatorio, por defecto `true`; `false` = dada de baja
`created_at`	`timestamptz`, por defecto `now()`

> Toda respuesta que devuelve una tarjeta agrega el campo calculado `disponible` = `limite_credito` − `saldo_utilizado`. En una tarjeta de débito el límite es siempre 0, por lo que su disponible también es 0: no representa el saldo de la cuenta asociada. Los importes de las tarjetas no tienen moneda propia; la aplicación web los muestra en guaraníes.

4. Requerimientos funcionales

RF-G2-01 — Listar tarjetas

`GET /api/v2/tarjetas`

Devuelve las tarjetas vigentes del sandbox, con la posibilidad de acotar por titular y por estado.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): `activa`, `bloqueada` o `vencida`.

Reglas de negocio

- Devuelve únicamente tarjetas no dadas de baja (`activo = true`), en cualquier estado.
- Los dos filtros se combinan entre sí. No hay filtro por tipo ni por marca.
- El filtro `estado=vencida` usa el estado guardado, no la fecha: una tarjeta `activa` con fecha de vencimiento pasada no aparece como vencida.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de tarjetas; cada elemento incluye las columnas de la tabla más el campo calculado `disponible`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` no es un entero positivo, o `estado` no es uno de los tres valores permitidos.

Fuente: `app/api/v2/tarjetas/route.ts`

RF-G2-02 — Emitir una tarjeta

`POST /api/v2/tarjetas`

Emite una tarjeta nueva de crédito o de débito para un cliente. La tarjeta nace activa y sin consumo.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `cuentaId` (opcional): número entero positivo de la cuenta asociada.
- `tipo` (obligatorio): `credito` o `debito`.
- `marca` (obligatorio): `visa`, `mastercard` o `amex`.
- `limiteCredito` (opcional): número mayor o igual a cero, como número JSON.
- `fechaVencimiento` (obligatorio): texto con el formato `YYYY-MM-DD`.

Reglas de negocio

- La tarjeta nace con `estado = 'activa'`, `saldo_utilizado = 0` y `activo = true`.
- En una tarjeta de **crédito** el límite es el `limiteCredito` enviado, o 0 si se omite.
- En una tarjeta de **débito** el límite es siempre 0: un `limiteCredito` enviado se ignora sin error.
- El `numero_enmascarado` lo genera el servidor con cuatro dígitos al azar; dos tarjetas pueden terminar con los mismos cuatro dígitos.
- El titular debe existir, y la cuenta también si se indica; un id inexistente es rechazado por la clave foránea.
- El formato de la fecha se valida con una expresión regular; una fecha con formato correcto pero inexistente (`2027-02-31`) pasa esa validación y es rechazada por la base como `EXECUTION_ERROR`.
- **No se valida que la fecha de vencimiento sea futura**: se puede emitir una tarjeta con vencimiento pasado, y nace `activa`.
- **No se valida que la cuenta pertenezca al titular, ni que esté activa**, ni que una tarjeta de débito tenga cuenta asociada.

Respuesta esperada

- `201 Created` con `data` conteniendo la tarjeta creada, incluido `disponible`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `tipo` o `marca` están fuera de la lista, `limiteCredito` es negativo o no es número, la fecha no tiene el formato `YYYY-MM-DD`, o el titular o la cuenta no existen.
`EXECUTION_ERROR`	400	La fecha tiene el formato correcto pero no existe en el calendario, o el límite excede el máximo que admite la columna.

Fuente: `app/api/v2/tarjetas/route.ts`

RF-G2-03 — Consultar una tarjeta

`GET /api/v2/tarjetas/{id}`

Devuelve el detalle de una tarjeta puntual, con su disponible calculado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la tarjeta en cualquier estado de negocio.
- Si no existe o está dada de baja, responde 404 con el mensaje "Tarjeta no encontrada.".

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta completa y `disponible`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/tarjetas/[id]/route.ts`

RF-G2-04 — Renovar una tarjeta

`PUT /api/v2/tarjetas/{id}`

Reemplaza la marca y la fecha de vencimiento de una tarjeta. El límite y el estado se manejan por sus propios endpoints.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `marca` (obligatorio): `visa`, `mastercard` o `amex`.
- `fechaVencimiento` (obligatorio): texto con el formato `YYYY-MM-DD`.

Reglas de negocio

- Es un reemplazo: los dos campos son obligatorios aunque solo cambie uno.
- `tipo`, `limiteCredito`, `estado`, `cuentaId` y el número no forman parte del cuerpo; si se envían, se ignoran.
- **No se valida el estado**: se puede renovar una tarjeta `bloqueada` o `vencida`.
- **Cambiar la fecha no cambia el estado**: una tarjeta `vencida` a la que se le pone una fecha futura sigue `vencida`, y no hay endpoint que la saque de ese estado.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo, la marca está fuera de la lista o la fecha no tiene el formato `YYYY-MM-DD`.
`EXECUTION_ERROR`	400	La fecha tiene el formato correcto pero no existe en el calendario.

Fuente: `app/api/v2/tarjetas/[id]/route.ts`

RF-G2-05 — Dar de baja una tarjeta

`DELETE /api/v2/tarjetas/{id}`

Da de baja lógica una tarjeta que no tiene consumo pendiente.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Una tarjeta con saldo utilizado mayor que cero no puede darse de baja: responde 409 con el mensaje "No se puede eliminar una tarjeta con saldo utilizado pendiente.".
- Con saldo utilizado en cero, marca `activo = false`; la fila permanece en la base.
- El estado de negocio no influye: se puede dar de baja una tarjeta `activa`, `bloqueada` o `vencida`.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`CONFLICT`	409	La tarjeta tiene saldo utilizado pendiente.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/tarjetas/[id]/route.ts`

RF-G2-06 — Bloquear una tarjeta

`POST /api/v2/tarjetas/{id}/bloquear`

Pasa una tarjeta activa al estado `bloqueada`.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `motivo` (opcional, en el cuerpo): texto libre. Se acepta pero no se guarda.

Reglas de negocio

- Solo se bloquea una tarjeta en estado `activa`.
- Si ya está bloqueada responde 409 con el mensaje "La tarjeta ya está bloqueada.": el bloqueo no es idempotente en el código de respuesta, para que el test distinga "la bloqueé yo" de "ya estaba bloqueada".
- Si está `vencida` responde 409 con el mensaje "Una tarjeta vencida no puede bloquearse.".
- El bloqueo no modifica límite, saldo utilizado ni fecha de vencimiento.
- El cuerpo es opcional: la request puede enviarse sin cuerpo.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta con `estado = 'bloqueada'`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`CONFLICT`	409	La tarjeta ya está bloqueada o está vencida.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo, o `motivo` no es texto.

Fuente: `app/api/v2/tarjetas/[id]/bloquear/route.ts`

RF-G2-07 — Activar una tarjeta

`POST /api/v2/tarjetas/{id}/activar`

Devuelve al estado `activa` una tarjeta bloqueada cuya fecha de vencimiento todavía no pasó.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo. No lleva cuerpo.

Reglas de negocio

- Solo se activa una tarjeta en estado `bloqueada`.
- Si ya está activa responde 409 con el mensaje "La tarjeta ya está activa.".
- Si su estado es `vencida` responde 409 con el mensaje "Una tarjeta vencida no puede activarse.".
- Si está `bloqueada` pero su fecha de vencimiento es anterior a hoy, responde 409 con el mensaje "La tarjeta está vencida: no puede activarse." y **el estado sigue en `bloqueada`**: no pasa a `vencida`.
- Una tarjeta que vence hoy todavía se puede activar.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta con `estado = 'activa'`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`CONFLICT`	409	La tarjeta ya está activa, está vencida, o su fecha de vencimiento ya pasó.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/tarjetas/[id]/activar/route.ts`

RF-G2-08 — Cambiar el límite de crédito

`PATCH /api/v2/tarjetas/{id}/limite`

Fija un nuevo límite para una tarjeta de crédito, sin permitir que quede por debajo de lo ya consumido.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `limiteCredito` (obligatorio, en el cuerpo): número mayor o igual a cero, como número JSON.

Reglas de negocio

- Solo aplica a tarjetas de crédito; sobre una de débito responde 409 con el mensaje "Solo las tarjetas de crédito tienen límite.".
- El nuevo límite no puede ser menor al saldo utilizado: responde **400** `VALIDATION_ERROR` con el mensaje "El límite no puede ser menor al saldo utilizado (N).". Un límite exactamente igual al saldo utilizado es válido y deja el disponible en cero.
- El disponible se recalcula con el nuevo límite; el saldo utilizado no cambia.
- **No se valida el estado**: se puede cambiar el límite de una tarjeta `bloqueada` o `vencida`.
- No hay tope máximo de negocio; el único techo es el que admite la columna en la base.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta con su nuevo `limite_credito` y `disponible`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`CONFLICT`	409	La tarjeta es de débito.
`VALIDATION_ERROR`	400	Falta `limiteCredito`, es negativo, no es un número JSON, o es menor al saldo utilizado.

Fuente: `app/api/v2/tarjetas/[id]/limite/route.ts`

RF-G2-TK — Obtener el token del grupo

`POST /api/v2/g2/auth/token`

Intercambia el usuario y la contraseña del grupo por un token JWT que puede usarse en lugar de la API key en cualquier ruta del curso. La ruta en sí también exige estar autenticado (API key del curso 2 o un token vigente).

Entradas y validaciones

- `username` (obligatorio): texto de 1 a 64 caracteres. El usuario del grupo es `c2_g02_tarjetas`.
- `password` (obligatorio): texto de 1 a 200 caracteres. Se entrega por el canal del curso; no se publica en este documento.

Reglas de negocio

- La contraseña se compara en la base de datos contra un hash bcrypt; nunca se devuelve ni se registra en claro.
- Usuario inexistente, contraseña incorrecta y credencial inactiva devuelven el mismo `400 VALIDATION_ERROR` con el mensaje "Usuario o password inválidos.": la respuesta no revela cuál de los tres falló. Es `400` y no `401` a propósito, para no confundirlo con el fallo de la API key.
- Si las credenciales son válidas pero pertenecen a otro grupo, la respuesta es `403 FORBIDDEN` con el mensaje "Estas credenciales pertenecen al grupo 3 y esta ruta es del grupo 2.".
- El token es un JWT firmado con HS256, emitido por `aiquaa-sandbox`, con vigencia de **3600 segundos**. Sus claims (`sub`, `usuarioId`, `grupo`, `curso`) son legibles: el token está firmado, no cifrado.
- El token identifica al grupo pero no restringe por grupo: con él se puede operar sobre los recursos de cualquier módulo del curso 2.
- La emisión no deja rastro en tablas de dominio (el curso 2 no tiene tabla de sesiones); solo queda la fila de auditoría.

Respuesta esperada

- `200 OK` con `data` conteniendo `token`, `tokenType` (`Bearer`), `expiresIn` (`3600`), `grupo` (`2`), `grupoNombre`, `curso` (`2`) y `usuario` (`id`, `nombre`, `email`, `username`).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `username` o `password`, están vacíos, o la combinación no es válida.
`FORBIDDEN`	403	Las credenciales son de otro grupo, o la API key usada para llamar es del curso 1.
`UNAUTHORIZED`	401	La request no trae API key ni token.

Fuente: `app/api/v2/g2/auth/token/route.ts`, `lib/group-token-route.ts` y `lib/jwt.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de cohorte (curso) → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Tipos en la respuesta:** los identificadores (`id`, `usuario_id`, `cuenta_id`, ...) viajan como **número**; los importes y tasas (`saldo`, `monto`, `limite_credito`, `tasa_anual`, ...) viajan como **texto con dos decimales** (por ejemplo `"5000000.00"`), por ser valores numéricos exactos de la base; las fechas sin hora viajan como `YYYY-MM-DD` y las marcas de tiempo (`created_at`, `fecha_pago`) en formato ISO 8601. Una aserción sobre un importe debe comparar contra texto o convertir a número antes.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo). Los campos no declarados en el esquema se ignoran sin error.
- **Coerción de tipos:** los identificadores (`id`, `usuarioId`, `cuentaId`, ...) se convierten de texto a número automáticamente, vengan por query, por ruta o en el cuerpo. Los **importes, tasas y plazos del cuerpo JSON deben enviarse como número**: enviarlos como texto (`"monto": "1000"`) devuelve `400 VALIDATION_ERROR`.
- **Cuerpo JSON:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body."; un cuerpo vacío se interpreta como `{}`. Un fallo del esquema devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid request." y el detalle del campo en `error.details`.
- **Reglas de negocio → 409:** cuando el cuerpo es válido pero el estado actual del recurso no permite la operación (saldo insuficiente, recurso en un estado que no admite la acción), la respuesta es `409 CONFLICT` con un mensaje en español que describe la regla. `400` queda para entradas inválidas.
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`, por ejemplo un `usuarioId` inexistente) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos (una fecha imposible, un número fuera de rango) cae en `400 EXECUTION_ERROR`. En los tres casos el `message` es el texto crudo de Postgres.
- **Importes con más de dos decimales:** la API no valida la cantidad de decimales; la base redondea a dos al guardar.
- **Operaciones que mueven dinero:** se ejecutan en una transacción con bloqueo de las filas involucradas; si alguna validación falla, no queda ningún cambio parcial (ni saldo modificado ni movimiento registrado).
- **Listados:** devuelven como máximo 100 filas ordenadas por identificador ascendente, sin paginación ni indicación de que hubo recorte. Los filtros opcionales se combinan entre sí (`AND`).
- **Headers de caudal:** toda respuesta que pasó el control de caudal incluye `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset` (este último en milisegundos Unix).
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera el framework y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** las requests quedan registradas en `public.sql_audit_log` con la credencial usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Sin control de titularidad:** la API no tiene roles ni noción de "mis" productos. Cualquier credencial válida del curso puede leer y modificar los recursos de cualquier cliente, y ningún endpoint verifica que la cuenta indicada pertenezca al `usuarioId` indicado.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso, y las acciones que cambian un estado sin crear un registro (bloquear, activar, aprobar, pagar una cuota, anular, cancelar).
`201 Created`	POST que crea un recurso o registra un movimiento de dinero nuevo.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Faltan credenciales, la API key no existe o está inactiva, o el token es inválido o venció.
`FORBIDDEN`	403	La credencial es válida pero es de otro curso, o las credenciales de un grupo se usaron en la ruta de token de otro grupo.
`RATE_LIMITED`	429	Se superaron las 30 requests por minuto de esa credencial.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe o ya está dado de baja.
`CONFLICT`	409	Regla de negocio violada sobre un recurso que existe, o violación de una restricción `UNIQUE`.
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/v2/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú muestra los módulos del grupo del alumno (más "Clientes", siempre visible) cuando su email figura en el roster del curso; si no figura, muestra todos. Cada listado arranca filtrado por el cliente con el que se inició sesión y permite cambiar a otro titular o a "todos"; la búsqueda, el orden y la paginación de los listados se resuelven en el navegador. **El front es deliberadamente más estricto que la API en varios puntos**: esas reglas están marcadas como "regla de UI" y solo se pueden probar por pantalla — la misma operación enviada directo a la API es aceptada. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
`/v2/tarjetas` — Listado de tarjetas	Tabla con tarjeta, titular, disponible, porcentaje de uso del límite, vencimiento y estado; filtros por titular, estado y tipo, y búsqueda por últimos cuatro dígitos o marca. Permite bloquear y activar desde la fila. Ejecuta RF-G2-01, RF-G2-06 y RF-G2-07.
`/v2/tarjetas/new` — Solicitar tarjeta	Formulario con titular, tipo, marca, cuenta, límite, mes y año de vencimiento y aceptación de condiciones, con una vista previa del plástico. Ejecuta RF-G2-02 y navega al detalle.
`/v2/tarjetas/{id}` — Detalle de tarjeta	Muestra el plástico, el estado, el disponible, el límite, lo utilizado y la barra de uso; permite bloquear o activar, cambiar el límite (RF-G2-08), renovar marca y vencimiento (RF-G2-04) y dar de baja (RF-G2-05).

- **Regla de UI:** una tarjeta de crédito nueva exige un límite entre 1.000.000 y 100.000.000 de guaraníes. La API acepta cualquier número mayor o igual a cero.
- **Regla de UI:** una tarjeta de débito debe estar asociada a una cuenta activa del titular. La API la acepta sin cuenta.
- **Regla de UI:** American Express solo se ofrece para crédito. La API acepta `amex` con `debito`.
- **Regla de UI:** el vencimiento debe ser posterior al mes actual y a no más de cinco años; la web envía siempre el último día del mes elegido. La API acepta cualquier fecha válida, incluso pasada.
- **Regla de UI:** el bloqueo exige elegir un motivo (robo, extravío, sospecha de fraude o bloqueo temporal). La API lo recibe como opcional y no lo guarda.
- **Regla de UI:** el límite solo se puede cambiar en una tarjeta activa. La API lo permite en cualquier estado.
- La web muestra como "Vencida" a toda tarjeta cuya fecha de vencimiento ya pasó, aunque su `estado` en la API siga siendo `activa`, y le oculta las acciones de bloquear y activar.
- El botón "Dar de baja" queda deshabilitado mientras la tarjeta tenga saldo utilizado.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G2-01 — Listar tarjetas

- Al listar sin filtros, la respuesta es 200 y cada elemento incluye el campo `disponible`.
- Al filtrar por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- Al filtrar por `estado=bloqueada`, todos los elementos devueltos tienen `estado` igual a `bloqueada`.
- Para cada tarjeta de crédito devuelta, `disponible` es igual a `limite_credito` menos `saldo_utilizado`.
- Al enviar `estado=cancelada`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G2-02 — Emitir una tarjeta

- Al emitir una tarjeta de crédito con `limiteCredito` 5000000, la respuesta es 201, `data.estado` es `activa`, `data.saldo_utilizado` es `"0.00"` y `data.disponible` es `"5000000.00"`.
- Al emitir una tarjeta de débito enviando `limiteCredito` 5000000, la respuesta es 201 y `data.limite_credito` es `"0.00"`.
- Al emitir una tarjeta de crédito sin `limiteCredito`, la respuesta es 201 y el límite es `"0.00"`.

- El `numero_enmascarado` devuelto cumple el formato `** **** ` seguido de cuatro dígitos.
- Al emitir con `marca=diners` o `tipo=prepaga`, la respuesta es 400 `VALIDATION_ERROR`.
- Al emitir con `fechaVencimiento` igual a `31/12/2028`, la respuesta es 400 `VALIDATION_ERROR`.
- Al emitir con `fechaVencimiento` igual a `2027-02-31`, la respuesta es 400 `EXECUTION_ERROR`.
- Al emitir para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se crea ninguna tarjeta.

RF-G2-03 — Consultar una tarjeta

- Dada una tarjeta existente, al consultarla la respuesta es 200 y `data.id` coincide con el solicitado.
- Dada la tarjeta sembrada 1 sin modificar (límite 10000000, utilizado 2500000), `data.disponible` es `"7500000.00"`.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Tarjeta no encontrada.".
- Al consultar con un identificador no numérico, la respuesta es 400 `VALIDATION_ERROR`.

RF-G2-04 — Renovar una tarjeta

- Dada una tarjeta existente, al enviar una nueva `fechaVencimiento` la respuesta es 200 y `data.fecha_vencimiento` refleja la nueva fecha en formato `YYYY-MM-DD`.
- El `limite_credito`, el `saldo_utilizado`, el `tipo` y el `estado` no cambian tras la renovación.
- Al enviar solo `marca` (sin fecha), la respuesta es 400 `VALIDATION_ERROR`.
- Al renovar una tarjeta inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G2-05 — Dar de baja una tarjeta

- Dada una tarjeta recién emitida, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, RF-G2-03 sobre esa tarjeta responde 404 y deja de aparecer en RF-G2-01.
- Dada una tarjeta con saldo utilizado, al intentar darla de baja la respuesta es 409 `CONFLICT` y la tarjeta sigue siendo consultable.
- Dar de baja dos veces la misma tarjeta devuelve 204 la primera vez y 404 la segunda.

RF-G2-06 — Bloquear una tarjeta

- Dada una tarjeta activa, al bloquearla la respuesta es 200 y `data.estado` es `bloqueada`.
- Al bloquear por segunda vez la misma tarjeta, la respuesta es 409 `CONFLICT` con el mensaje "La tarjeta ya está bloqueada.".
- Dada la tarjeta sembrada en estado `vencida`, al bloquearla la respuesta es 409 `CONFLICT`.
- Tras el bloqueo, `limite_credito`, `saldo_utilizado` y `disponible` conservan sus valores.
- Al bloquear una tarjeta inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G2-07 — Activar una tarjeta

- Dada una tarjeta bloqueada con vencimiento futuro, al activarla la respuesta es 200 y `data.estado` es `activa`.
- Al activar una tarjeta que ya está activa, la respuesta es 409 `CONFLICT` con el mensaje "La tarjeta ya está activa.".

- Dada una tarjeta emitida con vencimiento pasado y luego bloqueada, al activarla la respuesta es 409 `CONFLICT` y su estado sigue siendo `bloqueada`.
- El ciclo bloquear → activar → bloquear sobre una misma tarjeta responde 200 en los tres pasos.
- Al activar una tarjeta inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G2-08 — Cambiar el límite de crédito

- Dada una tarjeta de crédito con límite 5000000 y sin consumo, al fijar 8000000 la respuesta es 200, `data.limite_credito` es `"8000000.00"` y `data.disponible` es `"8000000.00"`.
- Dada una tarjeta con saldo utilizado 2500000, al fijar un límite de 2000000 la respuesta es 400 `VALIDATION_ERROR` y el límite no cambia.
- Dada una tarjeta con saldo utilizado 2500000, al fijar un límite de 2500000 la respuesta es 200 y `data.disponible` es `"0.00"`.
- Dada una tarjeta de débito, al cambiar su límite la respuesta es 409 `CONFLICT` con el mensaje "Solo las tarjetas de crédito tienen límite.".
- Al enviar `limiteCredito` negativo o como texto, la respuesta es 400 `VALIDATION_ERROR`.
- Al cambiar el límite de una tarjeta inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G2-TK — Obtener el token del grupo

- Con el usuario y la contraseña del grupo 2, la respuesta es 200, `data.tokenType` es `Bearer`, `data.expiresIn` es 3600 y `data.grupo` es 2.
- Una request posterior a cualquier endpoint del módulo enviando solo `Authorization: Bearer <token>` (sin `x-api-key`) responde con éxito.
- Con una contraseña incorrecta, la respuesta es 400 `VALIDATION_ERROR` con el mensaje "Usuario o password inválidos."; con un usuario inexistente, la respuesta es idéntica.
- Con las credenciales válidas del grupo 3 sobre esta ruta, la respuesta es 403 `FORBIDDEN`.
- Con un token alterado o vencido en el header `Authorization`, cualquier endpoint responde 401 `UNAUTHORIZED`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Reglamentación del BCP sobre tarjetas de crédito y medios de pago electrónicos	La emisión requiere identificar al titular, fijar un límite de crédito y entregar las condiciones del producto.	Parcial: la tarjeta queda asociada a un cliente y a un límite, pero no hay evaluación crediticia y la aceptación de condiciones existe solo en la pantalla web.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información y seguridad	El usuario debe poder bloquear su tarjeta de inmediato ante robo, extravío o fraude, y dejar constancia del motivo.	Parcial: el bloqueo es inmediato, pero el motivo no se guarda y no queda constancia de quién ni por qué bloqueó.
Estándar PCI DSS sobre protección de datos de tarjeta	El número completo y los datos sensibles de autenticación no deben almacenarse ni mostrarse en claro.	Cubierto por diseño: el sandbox solo maneja un número enmascarado ficticio y no existen código de seguridad ni PIN.
Normativa del BCP sobre límites de crédito y sobreendeudamiento	El consumo no puede superar el límite asignado, y los cambios de límite deben responder a una evaluación.	Parcial: la base impide que lo utilizado supere el límite y la API impide bajar el límite por debajo de lo utilizado, pero cualquier credencial puede subirlo sin tope ni evaluación.
Buenas prácticas de vigencia de medios de pago	Una tarjeta vencida debe dejar de operar automáticamente y renovarse con una nueva fecha.	No implementado: el vencimiento no se aplica solo, y renovar la fecha no saca a la tarjeta del estado `vencida`.

Brechas conocidas del sandbox

- Ningún endpoint modifica `saldo_utilizado`: no existen consumos ni pagos. El saldo utilizado solo proviene de los datos sembrados o de una preparación por el sandbox SQL.
- El estado `vencida` solo aparece en los datos sembrados: la API nunca lo asigna. Una tarjeta cuya fecha pasó sigue `activa` hasta que alguien intente reactivarla después de un bloqueo.
- No hay salida del estado `vencida`: no se puede bloquear, ni activar, y renovar su fecha con `PUT` no cambia el estado.
- `POST /tarjetas` acepta una fecha de vencimiento pasada y deja la tarjeta `activa`.
- `POST /tarjetas` no valida que la cuenta asociada pertenezca al titular ni que esté activa, y permite una tarjeta de débito sin cuenta.
- El `motivo` del bloqueo se acepta y se descarta: no queda guardado en ninguna tabla de dominio.
- `PATCH /limite` y `PUT /tarjetas/{id}` no miran el estado: operan igual sobre tarjetas bloqueadas o vencidas.
- El límite menor al saldo utilizado responde 400, mientras que las demás reglas de negocio del módulo responden 409: la convención no es uniforme.
- `PUT /tarjetas/{id}` permite dejar una tarjeta de débito con marca `amex`, combinación que la pantalla web no ofrece.
- El `numero_enmascarado` no es único: dos tarjetas pueden mostrar los mismos últimos cuatro dígitos.
- No existe control de titularidad: cualquier credencial del curso puede bloquear, activar, dar de baja o cambiar el límite de la tarjeta de cualquier cliente.
- El `disponible` de una tarjeta de débito es siempre 0 y no refleja el saldo de la cuenta asociada.